AI Case Sorter — Application Guide

Installing, setting up and running the desktop application (not the sorter hardware)

AI Case Sorter contributors

GPL-3.0-or-later

Table of contents

1. AI Case Sorter — Application Guide 4
1.1 Start here 4
1.2 Version dropdown 5
1.3 Download as PDF 5
1.4 Elsewhere 5
2. Download and Install 6
2.1 What you need 6
2.2 Windows 6
2.3 Linux and macOS 6
2.4 Starting the app 7
2.5 Where your data lives 8
2.6 Updating 8
2.7 PyTorch 8
2.8 When something goes wrong 8
3. Getting Started 9
3.1 What you see first 9
3.2 1. Connect the sorter 9
3.3 2. Pick the camera 10
3.4 3. Tune the crop 10
3.5 4. Choose how cases get classified 10
3.6 5. Route headstamps to slots 11
3.7 6. Sort 11
3.8 Where to go next 11
4. AI Case Sorter — User Guide 12
4.1 Contents 12
4.2 How this guide works 12
4.3 The window 12
4.4 Panels 14
4.5 Sort dashboard 16
4.6 Train 20
4.7 AI Config 21
4.8 Models 21
4.9 Community 22
4.10 Settings 23
4.11 Getting help 26

GPL-3.0-or-later

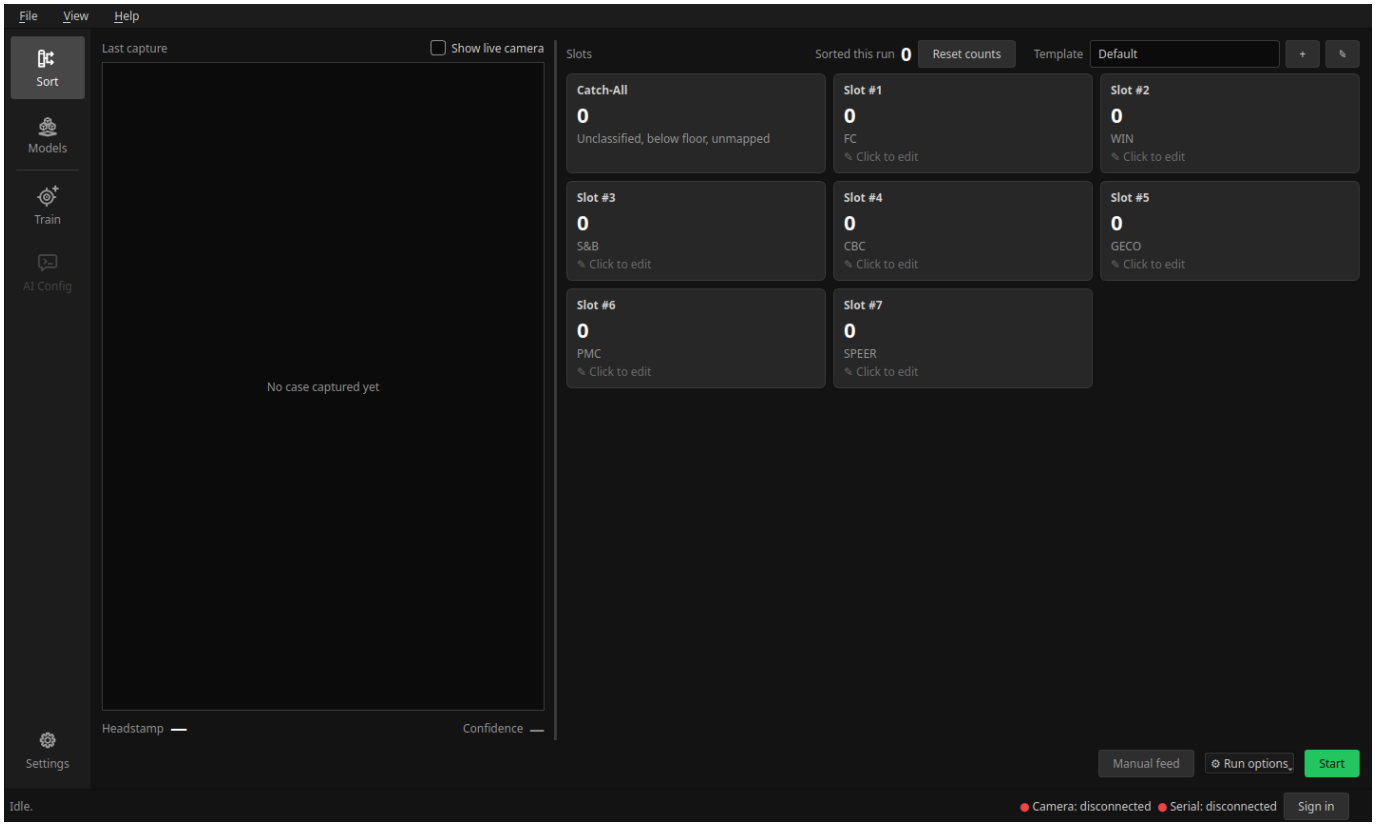
5. Troubleshooting 27
5.1 Where the logs are 27
5.2 An error flashed past in the status bar 27
5.3 Nothing happens when I start the app 27
5.4 The window won't open on Linux 27
5.5 The sorter is not detected 28
5.6 The camera preview is black 28
5.7 The crop is wrong, or the case isn't found 28
5.8 Everything lands in the Catch-All 28
5.9 Start is greyed out or refuses 28
5.10 Training fails or the machine runs out of memory 29
5.11 The app says a model update is available every time 29
5.12 An update fails its checksum 29
5.13 Resetting 29

GPL-3.0-or-later

1. AI Case Sorter — Application Guide

Documentation for the **desktop application** that drives the sorter. The machine itself is a separate project with its own build documentation; this is the software that runs it.

It sorts spent brass cartridge casings by **headstamp**: a camera photographs each case, an image classifier predicts the headstamp, and the serial-connected sorting machine drops the case into the right bin.



Two ways to classify:

- **Local model** — a PyTorch ConvNeXt model you trained yourself, downloaded from the community, or imported from a ZIP. Nothing leaves the machine.
- **AI Config** — send the cropped image to an OpenAI-compatible HTTP server.

Community features (model sharing, downloads, the feedback loop) are the only part that needs an account. Everything else runs signed out.

1.1 Start here

1. **Download and Install** — Windows installer, or from source on Linux and macOS.
2. **Getting Started** — from a fresh install to your first sorted case.
3. **Troubleshooting** — when the camera is black, the board isn't found, or everything lands in the Catch-All.

GPL-3.0-or-later

The **User Guide** covers every screen, in the order you meet them. It is also the guide the app itself shows on `F1`.

- The window — activity sidebar, status bar, menus
- Panels — serial monitor, classification history, themes
- Sort dashboard — slot cards, templates, running a sort
- Train — capture, label, train
- Models — activate, edit, import, export
- Community — browse and install published models
- Settings — camera, serial, image processing, AI config, theme
- Getting help — support package and updates

1.2 Version dropdown

This site is published per release. The selector in the header switches between them; `latest` always points at the newest stable release, and a release candidate is published under its own version without moving it.

1.3 Download as PDF

This documentation as a single PDF — built per version, so the PDF you download always matches the version you're reading.

1.4 Elsewhere

- Source, issues and releases on GitHub
- Download the latest release
- UI Modernization — the research and decisions behind the Qt client (for contributors)
- Contributing and Releasing

GPL-3.0-or-later

2. Download and Install

This page installs the **application** — the desktop program that drives the sorter. The machine itself is a separate project with its own build documentation; nothing here wires anything up.

Once it is installed, go on to Getting Started.

2.1 What you need

- **Windows, Linux or macOS.**
- **A webcam** — the app photographs each case with it.
- **The CS7.2 sorter** on a USB serial port, for actual sorting. Without it the app still runs: pick the **Emulated** port and everything except moving brass works.
- **Disk space for a model.** A trained checkpoint and its training images are hundreds of megabytes.

PyTorch is **not** in this list. It is only needed to train or run a model locally, it is large, and the app installs it for you the first time something actually needs it — see PyTorch below.

2.2 Windows

No git, no Python, no terminal.

1. Download `install-windows.bat` and `install-windows.ps1` from `installer/` into the same folder.
2. Double-click `install-windows.bat`.

It installs Python if you don't have it, puts the app in `%LOCALAPPDATA%\Programs\CaseSorter` (per-user — **no admin rights**), adds a Start Menu entry, and launches. The first launch downloads the app's dependencies and takes a few minutes; after that it starts immediately.

Windows will warn you the first time. The installer is unsigned, so Microsoft Defender SmartScreen shows "*Windows protected your PC*". Choose **More info → Run anyway**. Alternatively, right-click the downloaded file → **Properties** → tick **Unblock**, and it won't warn at all.

Re-running the installer is a safe way to repair a broken install. You do not need it for updates — see Updating.

Full details, options (`-InstallDir` , `-Version` , `-NoLaunch`) and the log locations are in `installer/README.md` .

2.3 Linux and macOS

2.3.1 With the installer

No git needed. In a terminal:

```
curl -fsSLO https://raw.githubusercontent.com/sjseth/AI-Case-Sorter-Py/main/installer/install-unix.sh
sh install-unix.sh
```

It downloads the latest release, checks it against the checksum GitHub publishes for it, and installs it in `~/.local/opt/ai-case-sorter` — per-user, **no** `sudo` . It then puts a launcher at `~/.local/bin/ai-case-sorter` .

Before it finishes, the installer sets up the app's dependencies in the same terminal: it fetches uv, the right Python and the packages the app needs, which takes a few minutes the first time. If a system library is missing (see the table below), this is where you are asked for your `sudo` password. The app itself is not started; when the installer is done, start it with:

```
ai-case-sorter
```


GPL-3.0-or-later

If that says *command not found*, `~/.local/bin` is not on your `PATH` — the installer warns you when that is the case. Run `~/.local/bin/ai-case-sorter` instead, or add the folder to your `PATH`.

Option	Effect
`--prefix DIR`	Install somewhere else, e.g. `/opt/ai-case-sorter` (the folder must be writable by you, so updates can replace it).
`--version TAG`	Install a specific release, e.g. `--version 2.3.0`. The default is the latest.
`--no-bootstrap`	Skip the dependency setup; the first start of the app does it instead.
`--force`	Install into a folder that already has other files in it, or replace an existing `ai-case-sorter` launcher the installer did not write.

Re-running the installer updates an existing install in place — a safe way to repair one. Your data is never touched. An older release pinned with `--version` may skip the dependency setup; the first start of the app does it instead.

2.3.2 From a checkout

The alternative, if you want git. The launch script installs uv if it isn't already there, uses it to fetch the right Python and the dependencies, and starts the app — every time, in one step.

```
git clone https://github.com/sjseth/AI-Case-Sorter-Py.git
cd AI-Case-Sorter-Py
./start.sh
```

2.3.3 System libraries

The one thing uv can't provide is system libraries. On a minimal Linux install the installer (or `./start.sh`) offers to install them with `sudo`; pass `--auto` to `./start.sh` (or set `AUTO_INSTALL=1` for either) to confirm automatically.

Library	Needed by	Debian/Ubuntu	Fedora	Arch
libGL	OpenCV	`libgl1`	`mesa-libGL`	`libglvnd`
glib	OpenCV	`libglib2.0-0`	`glib2`	`glib2`
libxcb-cursor	Qt's X11 plugin	`libxcb-cursor0`	`xcb-util-cursor`	`xcb-util-cursor`

The first two are required. The third is not — without it Qt falls back to Wayland, where a floating panel can't be moved or resized.

Either way, you need **some** Python 3 already on the machine, new enough to run `bootstrap.py` — 3.12 or newer. That is not the Python the app itself runs on: uv provisions that separately.

The same checkout works on Windows: `start.bat` instead of `./start.sh`.

2.4 Starting the app

- **Windows, installed:** Start Menu → **AI Case Sorter**.
- **Linux/macOS, installed:** `ai-case-sorter` in a terminal.
- **From a checkout:** `./start.sh` (Linux/macOS) or `start.bat` (Windows).

Every launch re-checks the dependencies against the lockfile, so pulling a change that adds one needs nothing extra from you.

The window opens on the Sort dashboard. A fresh install has nothing connected and nothing trained yet, so it shows a short setup panel rather than an empty slot grid — Getting Started picks up from there.

GPL-3.0-or-later

2.5 Where your data lives

Everything the app writes — models, training images, settings, logs — lives in one folder, **outside** the app directory:

Platform	Location
Windows	`%LOCALAPPDATA%\CaseSorter`
Linux	`~/.local/share/CaseSorter` (or `$XDG_DATA_HOME/CaseSorter`)
macOS	`~/Library/Application Support/CaseSorter`

File → Open data folder opens it. Keeping it separate is what makes updating safe: the updater replaces the app folder, and nothing of yours is in it. Deleting the folder resets the app to a fresh install.

Two overrides: set `CASESORTER_DATA_DIR` to put the data anywhere you like, or create an empty `portable.txt` next to `bootstrap.py` to keep it in `<app>/data` — for USB-stick installs.

2.6 Updating

Updates happen inside the app. It checks shortly after starting and offers what it finds in the status bar; **Help → Check for updates…** asks immediately.

Downloading only *stages* the update — nothing is replaced until you restart, and the button becomes **Restart now** when it is ready. The dialog's **Choose a different version…** lists every published release, older ones included, and optionally pre-releases. See Updates for the whole dialog.

- **Installed with an installer:** never re-run it for an update.
- **Running from a checkout:** `git pull` and launch again. The in-app updater still works, but a checkout is normally managed with git.
- Turn the automatic check off in the dialog, or set `CASESORTER_UPDATE_DISABLED=1`.

Installs older than 1.1.0 need one step in between. Versions 1.0.0 and 1.0.1 can't update straight to 2.x — their updater rejects the newer archive layout, and that check can't be patched remotely. Update to 1.1.0 first with **Choose a different version…**, then to the current release.

2.7 PyTorch

Training a model, or running one locally, needs PyTorch. The app asks the first time you do something that requires it and installs it into its own environment — an AI Config user is never prompted.

- **GPU:** an NVIDIA card with compute capability ≥ 8.0 (RTX 30-series or newer) is used automatically. Anything else falls back to CPU, which works and is slower.
- **AI Config mode needs no PyTorch at all** — the server does the classifying.

If you would rather install it yourself, `uv sync --extra ml` from a checkout does it, but read the CUDA-build note in the README first: the two routes ship different builds, and PyPI's Windows wheel is CPU-only.

2.8 When something goes wrong

Both halves of the launch leave a log under the data folder, in `logs/`: `install-<timestamp>.log` from the installer, and `launch.log` from every start of the app (the run before is kept as `launch.prev.log`).

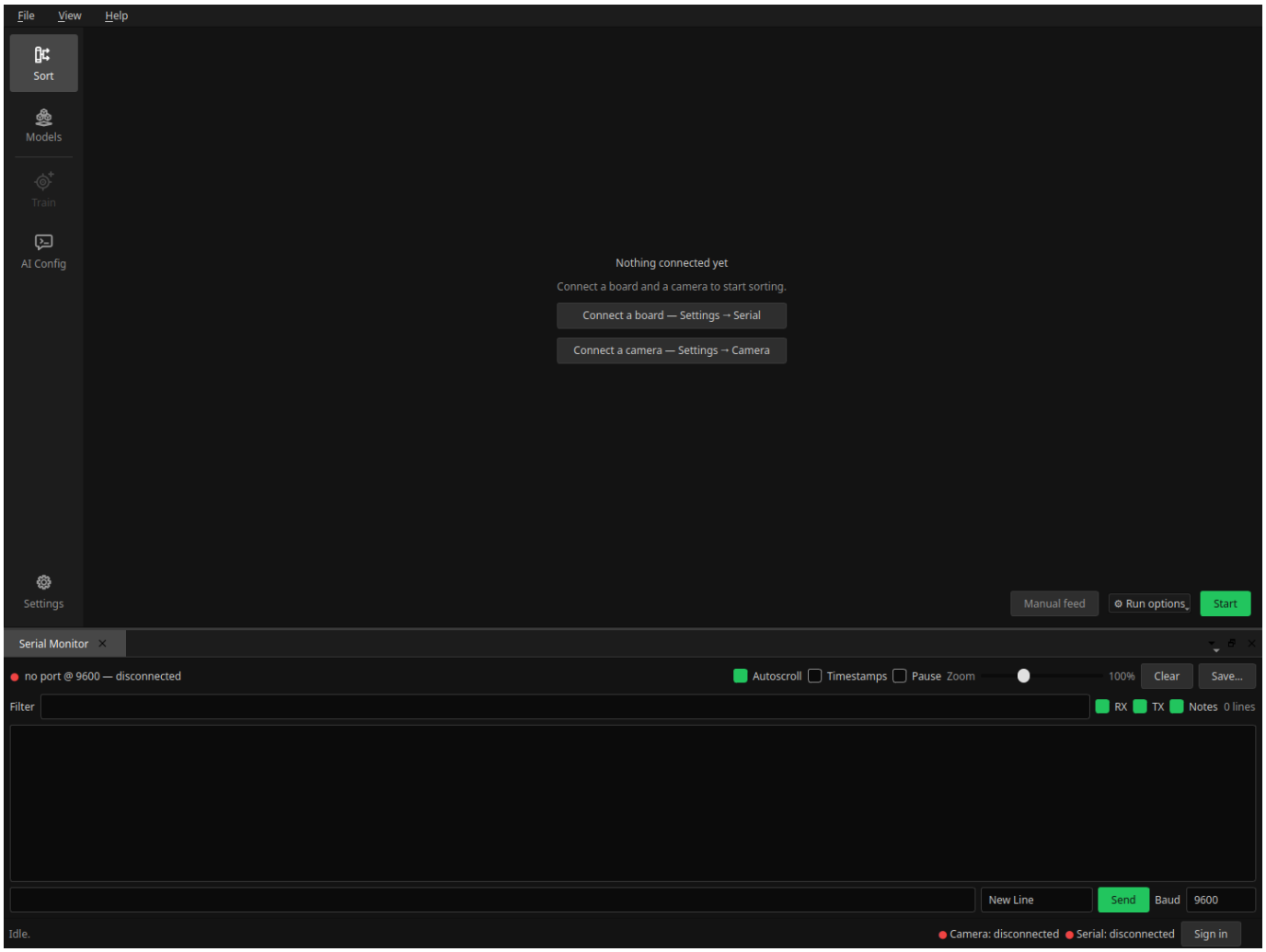
On Windows the console closes with the process and takes any traceback with it, so "*nothing happened when I clicked it*" is nearly always answered by `launch.log`. Attach both when reporting a problem — see Troubleshooting.

GPL-3.0-or-later

3. Getting Started

From a freshly installed app to your first sorted case. Each step links into the User Guide, which is the reference for the screen once you know why you're on it.

3.1 What you see first



The window is the same on every screen: **activity buttons** down the left, your working screen in the middle, movable panels around it, a menu bar on top, and a **status bar** underneath carrying the camera and serial indicators and the sign-in.

The app opens on the Sort dashboard — where sorting eventually happens. On a fresh machine nothing is connected and nothing is routed to a slot yet, so it shows a short setup panel with buttons straight to the two settings pages you need first, rather than an empty slot grid.

Nothing here needs an account. Signing in is only for the Community screen.

3.2 1. Connect the sorter

Settings → Serial. Pick the port the board is on and press **Connect**. The status bar's ● **Serial** indicator turns green and names the port, speed and the firmware version it handshook with.

If no port appears, the board is probably not the problem — see Troubleshooting.

GPL-3.0-or-later

No hardware yet? Pick the **Emulated** port. The built-in emulator speaks the real board's protocol, so every screen, every setting and the whole run loop work; only brass doesn't move. It is the right way to explore the app before the machine is built.

The same page holds the **board init settings** — feed and sort speeds, homing offsets, motor current, the camera LED. Those belong to the machine, and the sorter's own build documentation is what tells you what to put in them. Tick **Initialize these settings on startup** once they're right, and the app pushes them on every connect.

3.3 2. Pick the camera

Settings → Camera. Press **Detect / refresh**, choose the camera and a resolution, then **Apply**. The preview underneath updates immediately, so a wrong pick is obvious before you leave the page.

Nothing grabs the camera on its own — both actions are yours.

3.4 3. Tune the crop

Settings → Image Processing. The classifier never sees the raw frame: every case is cropped to a 480×480 image of the headstamp first, and a model recognises brass that looks like what it was trained on.

Press **Capture** to grab one frame and tune against it — every change re-processes that same frame, so you are not feeding a case per adjustment. Start with the **minimum and maximum case radius**: they bracket the size of the case in the frame, and most bad crops are one of those two being wrong.

These settings belong to the **active model**, not to the app, because case diameter and primer size are properties of the cartridge. Switching models brings its own values back.

3.5 4. Choose how cases get classified

This is the one real decision, and the Models screen is where you make it. Exactly one row there is ● **ACTIVE**, and that row alone decides how classification happens.

A fresh install starts in **AI Config mode** with an empty starter model ("Default", 9mm) sitting in the library, unused. Three ways forward:

Route	Do this	Needs
Someone else's model	Community → pick one → **Download model**, then **Activate** it on Models.	An account; PyTorch (offered when needed)
Your own model	Models → **New model**, activate it, then Train: feed, label, save, repeat — and train when you have enough images.	PyTorch (offered when needed)
An HTTP server	Leave **"Use AI Config"** active and fill in the server on AI Config.	An OpenAI-compatible endpoint

The **Train** and **AI Config** buttons in the sidebar follow that choice — one is live, the other dimmed, and a dimmed one still opens a page explaining which mode you're in. A community model dims both: its publisher trained it.

If you already have a model from the Windows app, **Import…** on the Models screen reads its ZIP directly.

PyTorch is installed on demand. Training and local inference need it. The app offers to install it the first time you do something that requires it, and never before — see PyTorch.

GPL-3.0-or-later

3.6 5. Route headstamps to slots

Back on the Sort dashboard, each slot on the machine is a card. Click one and tick the headstamps that should land in it.

- **Slot 0 is the Catch-All** and can't be edited. Everything unrouted, unrecognised, or below the confidence floor goes there.
- A headstamp belongs to one slot at a time (outside package mode).
- Your layout is saved as you make it, under the template named in the grid's header — there is no "save" step. Make a second template when you want a second arrangement.

⚙ **Run options** is worth one look before the first run: the **confidence floor** is the percentage a prediction must reach to be trusted, and **Automatically select trays** routes a confident headstamp with no slot to the first empty one.

3.7 6. Sort

Manual feed runs a single cycle — feed, capture, classify, sort. Use it a few times first: it shows the crop, the label and the confidence for one case, which is where a bad crop or a mismatched model shows up cheaply.

Start then runs the loop continuously and turns into a red **Stop**. Case counts survive a stop, so you can clear a jam and pick up mid-tray.

If Start is greyed out or refuses, it says why: no board connected, no API key or model name in AI Config mode, a missing checkpoint, PyTorch not installed yet, or an unread moderator note.

Watch the run in the **Classification History** panel (**View** menu) — one tile per case, with the crop, the headstamp, the confidence and the bin it went to.

3.8 Where to go next

- The **User Guide** — every screen and control in detail. It is also in the app: `F1` opens it at the section for the screen you're on.
- **Troubleshooting** — when the camera is black, the board isn't found, or everything lands in the Catch-All.
- **Help → Export support package…** collects your configuration into a shareable report when you need to ask someone.

GPL-3.0-or-later

4. AI Case Sorter — User Guide

A guide to the desktop client, written for the person running the machine. It covers every screen and panel you use day to day, in the order you meet them.

4.1 Contents

- The window — the activity sidebar, the status bar and the menus that frame everything else.
- Panels — the movable side and bottom panels: Serial Monitor, Classification History, Catch-All, the guide you are reading, Themes and Messages.
- Sort dashboard — the main working screen: the current case, the slot cards, sorting templates and the run controls.
- Train — capture cases, label them, and train a model from them.
- AI Config — the other way to classify: an HTTP server instead of a local model.
- Models — the model library: activate, edit, import, export.
- Community — browse and install published models.
- Settings — Camera, Serial, Image Processing, Theme and Import from Windows.
- Getting help — this guide, the support package and updates.

4.2 How this guide works

This whole guide is one Markdown file, rendered two ways:

- **On GitHub**, browsing straight to `docs/guide/GUIDE.md`.
- **In the app**, in the User Guide panel (`F1`, or Help → User Guide), which opens this file and jumps straight to the section for whatever screen you're on — like Settings → Serial.

Press `F1` again after moving to another screen and the panel follows you. The **< Back** button at the top of the panel returns to the previous section.

4.3 The window

The window is the same everywhere: a column of activity buttons down the left, your working screen in the middle, movable panels around it, a menu bar on top and a status bar underneath.

GPL-3.0-or-later

4.3.1 Activities

The sidebar switches between the things you do, one button each. A thin line splits it in two: the screens that are always live above it, and the pair that follows the active model below it.

Button	What it is
Sort	The Sort dashboard — where sorting happens.
Models	The model library.
Community	The community catalogue. Shown only while signed in.
(separator line)	Below it, the two ways a classifier is taught.
Train	The Train screen — teaching a local model of your own.
AI Config	The AI Config screen — the equivalent step when an HTTP server does the recognising: teaching *it* what to look for.
(separator line)	Below it, the app itself rather than a way of working.
Settings	Everything you configure once and rarely touch again.

Train and AI Config are always both there, and exactly one of them is in use at a time — which one follows the **active model** (see Models):

- **a model you own** — Train is live, AI Config is dimmed;
- **AI Config mode** ("Use AI Config" on the Models page) — AI Config is live, Train is dimmed;
- **a community model** — neither is live: its publisher trained it, and it, not a server, does the classifying.

Hover either button and the tooltip says which of those you are in. A dimmed button still works — it is the screen behind it that explains the state, and carries a button to the Models page to change it. Never a dead end.

4.3.2 Status bar

Along the bottom, from left to right:

- **Messages** — what the app just did ("Auto-connected to COM3.", "Run stopped."). This is where a refused action explains itself. It shows one line, cut off at the window's edge, until the next replaces it — **click it** to open the Messages panel, which keeps the recent ones in full.
- ● **Camera** and ● **Serial** — connection indicators. Green means connected, and the serial one names the port, speed and the firmware version it handshook with.
- **Inference: MPS · Apple M4** (for example) — where classification runs when a local model is active: a CUDA GPU, an Apple GPU (MPS), or the CPU, with the hardware's name. Appears shortly after startup once the model's engine has loaded (or after the first classification if PyTorch was just installed). Absent in AI Config mode, where classification is an HTTP call to your configured server rather than a local computation.
- **Update to *version*** or **Restart to update** — appears only when there is an app update to fetch or a downloaded one waiting. See updates.
- **Model update: v*N*** — appears when the active community model's publisher has released a newer version. See community model notices.
- **Your name** and **Sign in / Sign out** — the community account. Nothing outside the Community screen needs one.

GPL-3.0-or-later

4.3.3 Menus

- **File → Open data folder** opens the folder holding the database, your models, their training images and the logs. **Quit** closes the app.
- **View** switches each panel on or off, and holds **Re-dock panels**. `Ctrl+Shift+M` toggles the Serial Monitor.
- **Help** holds this guide (`F1`), Check for updates…, Export support package…, About and License.

4.4 Panels

Six panels can sit around your working screen. Each one can be moved, tabbed together with another, torn off into its own floating window, or closed:

- **Serial Monitor** — along the bottom, open by default.
- **Classification History** — on the right, closed by default.
- **Catch-All** — on the right, closed by default.
- **User Guide** — on the right, closed by default (this guide).
- **Themes** — on the right, closed by default.
- **Messages** — on the right, closed by default.

Moving a panel: drag it by its *tab* — the small labelled tab at the edge of the panel, not its title. As you drag, blue drop indicators appear showing where it can land: the four edges of the window, or the middle of another panel to tab the two together. Drop it outside the window and it becomes a floating window you can put on a second monitor.

Getting a panel back: View → Re-dock panels returns every open panel to where it started, un-floated. Use it any time a panel ends up somewhere you didn't intend — it always works, which dragging one back does not.

Closing and re-opening: the × on a panel closes it; its entry in the **View** menu switches it back on. Your arrangement is remembered and restored the next time you start the app.

4.4.1 Serial Monitor

Live traffic between the app and the board — every line it sends and every line the board answers, in the order it happened. It is the first place to look when the machine does something unexpected.

`Ctrl+Shift+M` opens it and closes it again, from anywhere in the window — including while you are typing in its own command box.

- The header shows the connection state, plus **Autoscroll**, **Timestamps** and **Pause**. Pausing holds new lines back and releases them when you un-pause; nothing is lost.
- **Zoom** scales the text from 50% to 200%, for reading the log across the bench.
- **Clear** empties the view. **Save…** writes what you are currently looking at to a text file — filtered lines are not written, so narrow the view first if that's what you want to send. To keep the whole session on disk as it happens, tick **Log serial traffic to a file** on Settings → Serial.
- **Filter** hides every line that doesn't contain what you type, and the **RX** / **TX** / **Notes** toggles hide a whole direction (received, sent, and the monitor's own commentary). Filtering only hides: clear the box and the traffic is all still there.
- The bottom row sends a command by hand — type it, pick the line ending the firmware expects, press **Send**. `Up` and `Down` walk back through what you have sent before.
- **Baud** changes the port speed and reconnects at it. It is the same setting as Settings → Serial's; change it in either place and the other follows.

4.4.2 Classification History

A grid of tiles, one per sorted case: the cropped image, the headstamp, the confidence, the bin it went to, and a running case number.

GPL-3.0-or-later

Tiles never scroll or move. When the grid is full the newest case overwrites the oldest tile in place, and a coloured border trails the most recent few so you can see where "now" is. That is deliberate — it means you can watch one position and see cases go past, instead of chasing a scrolling list.

The panel holds **however many tiles fit it**, and it recounts whenever its size changes: widen the window or drag the panel wider and there are more columns; narrow it and the tiles that no longer fit are dropped, oldest first. Nothing is ever hidden behind a scrollbar — what you can see is all of it, which is what makes watching one position work.

Zoom (at the bottom of the panel, 50–200%) sets the tile size, and feeds the same count: bigger tiles are easier to read across a bench, smaller tiles mean more of them fit. Click any tile to open that case's image full size.

4.4.3 Catch-All panel

Slot 0 is the Catch-All: cases the run could not put in a bin. Open the panel from **View → Catch-All breakdown**, or by clicking the Catch-All card on the slot grid. It shows how many cases went there this session, which headstamps they were, and why.

The header is that count against everything sorted so far ("319 in catch-all of 615 sorted (52%)"). It stays the number of cases physically in the bin, even after you assign some of them.

Top 10 and **ALL**, at the upper right, choose how much of the list you see. Exactly one is lit, in a neutral highlight; the other stays an ordinary button. Both keep a visible outline. The panel opens on **Top 10**. Each row is a headstamp that would still land in the catch-all if it were seen now, with its remaining count, its share of the catch-all, and the reason:

- **Below floor** — the prediction did not reach the confidence floor. This is the only reason drawn in the warning colour.
- **Unassigned** — the headstamp is in the model, and its slot is the Catch-All.
- **Unknown** — the label was empty, or it is not one of this model's headstamps.
- **Upside down** — the case was classified as upside down, which has no bin of its own. That is a normal trained class, not a failure.
- **Batch full** — package mode, and every slot for that headstamp was already full.

Top 10 keeps the ten fullest of those rows. Anything past them is one greyed **Other** row ("Other: 2 headstamps, 4 cases"). It cannot be selected. **ALL** lists every such headstamp, in the same order, with no Other row. The table scrolls when that list is longer than the panel. Opening the panel, and switching between the two, leaves the panel the size you last had it. The lit choice lasts until you quit.

A headstamp grouped under a parent is listed as the parent; point at the reason and the tooltip names the child labels and the full count of each reason.

Giving a headstamp a slot — from this panel, from a slot card, or from **Assign by headstamp** — takes it off that list, and the next one moves up. That only removes the **Unassigned** cases, and an **Unknown** label that now has a slot. **Below floor**, **Upside down** and **Batch full** stay, because a bin does not fix them; a row that was a mix of those and unassigned cases shrinks to the part that is left. Take the slot away again and the headstamp comes back at its full count.

Under the table, a line keeps what left the list: "Assigned this session: BPS → #6 (42 already in bin 0), IK → #7 (17)". Those cases are still in the Catch-All bin; the next run is what puts them in the new slot. **Reset counts** clears the line with the rest of the tally.

Assign {name} to empty slot #N, under the table, puts the selected headstamp in the first empty bin and then selects the next headstamp that can still be assigned, so you can work down the list. It does this for any row, including one that only **ALL** shows. The button is unavailable, and says why, when nothing is selected, when there is no empty slot, when the label is unknown, or when it already has a slot — the button then reads **→ #N**. A row that is only below the confidence floor, for a headstamp that already has a slot, is the same: those cases missed the floor, and assigning them again would not move them.

Add to existing slot…, beside that button, lists the bins that already have brass ("#4 WMA, WMA NATO"), in slot order. Pick one and the selected headstamp shares that bin with whatever is already there — useful when a headstamp shows up rarely and does not need a bin of its own. It is unavailable, and says why, when nothing is selected, when the label is unknown, when it already has a slot, or when no slot has a headstamp yet. Assign to empty slot stays the main button. Either choice selects the next headstamp that can still be assigned. The menu stays open while cases keep arriving, so a bin can be picked during a run; the table catches up when the menu closes.

GPL-3.0-or-later

You can assign during a run. The case already in the wheel still drops in the Catch-All; put an empty bin in the slot you just filled, and the next case of that headstamp uses it. In parent mode the assignment is the parent's slot, which is the slot the run actually reads.

The counts follow the slot cards: they survive Stop and Start, and **Reset counts** clears them together with the cards.

4.4.4 Themes panel

The whole list of themes in one place. Click one and the app repaints immediately, so you can try them against the room's lighting without leaving what you were doing. **Edit theme…** opens the theme editor.

This is the same list as Settings → Theme; whichever you use, the other follows.

4.4.5 Messages

Every message the status bar has shown this session, newest at the bottom, each with the time it appeared — the last 200 are kept. Open it from **View → Messages**, or by clicking the message in the status bar.

This is where to read an error in full. The status bar cuts a long message off at the window's edge and replaces it with the next one; here it is kept whole, wrapped to the panel's width. Errors are marked `[error]` and printed in bold in the theme's error colour.

A step that is still in progress ("Connecting to COM3…", "Classifying…") is replaced by whatever comes next, so a long run shows its latest step rather than four lines per case. An error is never replaced: the step it interrupted stays just above it.

- Select any text and copy it as usual, or press **Copy all** to copy the whole log — timestamps and `[error]` marks included — for a bug report.
- **Clear** empties it.

The panel is not saved: it starts empty each time the app does, so copy what you need before quitting.

4.5 Sort dashboard

The Sort activity (the sidebar's top button) is where sorting actually happens. It combines the current case, the slot layout and the run controls on one screen.

On a fresh machine — nothing connected and nothing routed to a slot yet — this screen shows a short panel with buttons straight to Settings → Serial and Settings → Camera instead of an empty grid.

4.5.1 The current case

The left-hand panel shows the **last captured and cropped headstamp** — exactly the image the classifier was given — with what it made of it underneath: the headstamp it matched and how confident it was. A confidence below the confidence floor is coloured as a warning, and that case goes to the Catch-All. Only the current case is shown here; the running record is in the Classification History panel.

Show live camera above the panel adds the live camera feed as a smaller second panel below the crop. It is off by default — the feed is a setup aid, not what an operator watches during a run — and while it is off no frame is fetched or drawn. If the camera isn't running, clicking the feed takes you to Settings → Camera.

GPL-3.0-or-later

4.5.2 Slot cards

Every slot on the machine gets a card, arranged in a grid. Slot 0 is the **Catch-All**, for anything unclassified, below the confidence floor, or not routed to a slot; the rest are your bins. A card shows:

- the slot number (or "Catch-All")
- how many cases have landed there this run
- every headstamp currently routed to it, listed in full — the card grows to fit the list rather than truncating it

Click the **Catch-All** card (it says "🔍 Click for breakdown") to open the Catch-All panel. The other cards say "✎ Click to edit" and open the assignment editor.

Above the grid: **Assign by headstamp…** opens the headstamp-first view of the whole layout, **Sorted this run** counts every case this run has sorted, **Reset counts** zeroes the counters (the grid's and the per-bin ones) without touching your assignments, and the template picker names the layout the cards are showing.

4.5.3 Editing an assignment

Click a numbered slot's card to open its assignment editor. The Catch-All card opens the Catch-All panel instead — slot 0 is not a bin you configure from that editor. Tick a headstamp to route it to the slot; unticking sends it back to the Catch-All. Outside of package mode a headstamp can only be assigned to one slot at a time — ticking it here moves it off whichever slot it was in before, and the row tells you which one that was.

The list opens in three groups, alphabetical within each: what is already in this slot, then headstamps not in any slot yet, then those routed to another slot. Working through a big model, the unassigned part shrinks every time you reopen a card. In package mode, where a headstamp can fill several slots, "routed to another slot" means any other slot, and the row lists which. The order is fixed while the editor is open — ticking a row doesn't move it — and is worked out again when you change the filter.

The filter box has the cursor as soon as the editor opens. It matches every word you type, in any order and ignoring case, so `win 9` finds `WIN 9MM LUGER`. When the filter leaves exactly one row, **Enter** ticks it (or unticks it, if it was ticked); otherwise Enter does nothing. Close the editor with **Esc** or **Close**.

4.5.4 Assigning by headstamp

A card's editor answers "what goes in this bin?". Laying out a big model from scratch is the opposite question, asked once per headstamp: "which bin does this one go to?". **Assign by headstamp…**, above the slot grid, lists every headstamp in one table with the slot it is routed to (**—** means the Catch-All), so the whole layout is one pass at the keyboard:

1. Type part of a name. The filter works as in the card editor — every word, any order, ignoring case, so `win 9` finds `WIN 9MM LUGER`.
2. Press **Enter** or **↓**. The cursor moves into the table, on the top match.
3. Type the slot number. It is saved straight away, and the slot cards behind the window update as you go.
4. Type the next name — any letter typed in the table starts a new search.

0, **Delete** or **Backspace** sends a headstamp back to the Catch-All. The arrow keys move between rows, and double-clicking a slot (or **F2**) opens a number box for anyone who prefers the mouse.

On a machine with 10 or more slots, a number that could still be the start of a longer one waits for you: with 16 slots, `1` waits for a second digit (`12`) or **Enter** (slot 1), while `2` to `9` are saved at once. **Esc** or **Backspace** takes a half-typed number back. A number the machine doesn't have is refused, with a note under the table.

Like the card editor, this follows the routing the run will use. With parent classifications on, parent groups carry the slot and a **Contains** column lists each group's headstamps — the filter searches those too, so typing a headstamp's name finds the group it belongs to. In package mode, where a headstamp can fill several slots, typing a slot number adds that slot to the row or takes it off again, and **0**, **Delete** or **Backspace** clears them all. Every change lands in the active sorting template, just as a card edit does.

GPL-3.0-or-later

4.5.5 Sorting templates

The **Template** picker on the slot grid's header row holds the active **sorting template** — a named snapshot of the whole slot layout, so one model can carry several bin arrangements ("Range brass" vs. "Match prep") and switch between them. **+** creates one (optionally copied from the current layout); ✎ renames or deletes the active one.

You never have to save a template: the active one follows your edits as you make them. Switching templates replaces every slot assignment at once, so the whole picker is blocked while a run is in progress — stop first. Standard and package mode keep separate template lists, because their layouts mean different things.

4.5.6 Run options

⚙ **Run options** on the strip at the foot of the page holds everything that changes how a run behaves:

- **Store images** — keep a copy of each run's captures (none / above the floor / below it / all), under the active model's folder, so AI Config mode stores nothing. Anything but "None" will use real disk space over a long session.
- **Confidence floor** — the percentage a prediction has to reach to be trusted. Below it, the case goes to the Catch-All.
- **Automatically select trays** — when a confident prediction has no slot, route it to the first empty one and keep the assignment. With parent classifications on, that assignment is the parent's slot, which is what the run actually uses — assigning the child on its own would leave the cases in the Catch-All. A label the model does not have is left alone.
- **Package mode** and **Batch size** — see package mode.
- **Feed timeout** — how many seconds the feed sensor may go without brass before the run decides the hopper is empty and drains the wheel (default 6 s). Raise it if your collator pauses for long stretches and the run keeps draining in the middle of a tray. See running out of brass.
- **Run continuously** — after that drain, keep the run going instead of ending it: the machine waits for brass on the feed sensor and picks back up by itself when you refill the hopper. Press **Stop** to end the run.

4.5.7 Start, Stop, and Manual feed

The strip at the foot of the page, with the green **Start** at the far right:

- **Start** begins the continuous sort loop: capture, classify, sort, repeat. It is one button — while the loop runs it reads **Stop**, in red.
- **Stop** ends it. Case counts are kept, not reset, so you can clear a jam and pick back up mid-tray.
- **Manual feed** runs exactly one cycle without starting the loop, useful for testing a single case.

If the board stops responding while a run is going — the USB cable knocked out, the board losing power, the adapter re-enumerating — the run **stops** and a message says so. The serial indicator in the status bar turns red at the same moment, and the Serial Monitor records what went wrong. The app does not reconnect by itself and does not offer to resume: once the link is gone, where the wheel is and which case is about to drop are no longer known, and carrying on from a guess is how a case ends up in the wrong bin. Fix the cable, then reconnect from Settings → Serial and press Start again.

Without a board connected, Start and Manual feed are greyed out and say so. Otherwise starting is refused, with a message explaining why, if the AI Config API key or model name is unset, the active model's checkpoint is missing, PyTorch isn't installed yet for a local model, the model needs a newer PyTorch than this machine has (see below), or a moderator note is waiting to be read.

"This model needs a newer PyTorch." A model can only be read by the version of PyTorch it was trained with, or a newer one — never an older one. So a model trained on someone else's up-to-date machine, or on yours before an older PyTorch was installed, may refuse to load here. The message names both versions: the one the model needs and the one you have. Update PyTorch when the app offers to, and it will load. Nothing about this is recoverable by retrying, and a model that says this has not been damaged — it is simply newer than the software trying to open it.

"Update PyTorch to use this model." A different message, and a security one. A trained model file from somewhere else can carry code, and PyTorch versions older than 2.10.0 have a flaw that lets such a file run it. So a model file that **was not trained on this machine** — one downloaded from the Community, imported from a ZIP, or brought across from the Windows app — is

GPL-3.0-or-later

not loaded until PyTorch is updated, and there is no "not now". A model you trained here only *offers* the update. Models that were already in your library before this check arrived count as trained here, so nothing you were already sorting with changes. A fresh install already has a new enough PyTorch, so most people never see this. The check runs for every case, not just at Start: activating such a model on the Models page in the middle of a run stops the run with this message instead of loading it. Training the model here replaces its file with your own, after which it counts as trained here.

4.5.8 Running out of brass

When the collator runs dry mid-run, up to three cases are still riding the feed wheel — fed past the sensor but not yet dropped. The app notices the board waiting on brass that isn't coming (the **Feed timeout** in Run options — six seconds of quiet at the feed gate unless you change it), confirms nothing is mid-fall, and then **flushes the wheel**: each remaining case is walked to the camera, classified, and dropped into its own slot, exactly as if the hopper were still full. The run then ends on its own with the wheel empty, and the status bar says how many in-flight cases were placed.

With **Run continuously** on, the run does not end there. Once the wheel is empty it waits on the feed sensor — the status bar says "waiting for brass" — and as soon as a case reaches the sensor it is fed to the camera and the run resumes at full speed, with the same counts and the same sort run. Refill the hopper whenever you like; **Stop** ends the wait.

A few details worth knowing:

- If brass keeps arriving during the flush — the "empty" was really the collator pausing — the run notices and resumes at full speed by itself, whether or not Run continuously is on. Topping up the hopper mid-run is fine. (During the flush the board feeds without consulting the sensor, so it is the camera seeing new brass arrive that triggers this.)
- This relies on the board's **feed sensor**. With the sensor disabled the firmware feeds blind, the app can't tell an empty hopper from a full one, and a run at the end of the brass classifies empty air until you press Stop — exactly as before.
- Knowing when the wheel has finished emptying relies on telling a case at the camera from an **empty pocket** — and an empty, in-focus pocket still looks like a circle to the detector. The tiebreaker is the **Case brightness floor** in Settings → Image Processing: brass has to read brighter than it. Tune it once for your machine — capture the empty nest, read the "disc brightness" number in the status line, set the floor above it, then capture a case and confirm it reads clear of the floor. A shiny, worn pocket can read bright enough to need a floor well above the default; if the floor is set wrong the run tells you so and stops rather than cycling the machine forever.

A jam during the flush stops it immediately; whatever is still in the wheel stays there for you to clear. Deliberately so: at the end of the brass the app never feeds blind to shake a case loose, because that is how a case ends up in the wrong bin.

4.5.9 Package mode

Turning on **Package mode** in Run options switches the grid to batch counting against the one **Batch size** you set, shown as "count / target" on every slot card. A slot that reaches the target stops taking that headstamp; once every slot for a given headstamp is full, the run halts and asks you to empty bins and reset counters. A slot card's ⟲ **Reset** button empties just that bin's counter without stopping the run, so it can keep filling.

4.5.10 Community model notices

When the active model came from the Community and you are signed in, the app checks in with its publisher each time you open the Sort screen. Two things can come back, and neither stops a run in progress:

- **Model update: v*N*** in the status bar — a newer version has been published. The button opens a summary of what you have versus what is available; **Update now** installs it in place, keeping your slot assignments, sorting templates and the name you gave it. Dismissing it is fine — it comes back next time you open Sort.
- **Moderator notes (*N*)** on the run strip — messages from the model's moderators, usually about the feedback images the model is collecting. A note you haven't acknowledged opens by itself and **blocks Start** until you have read it. The same button is the history of every note, including the ones already acknowledged.

If the check can't reach the server, nothing appears and the app behaves exactly as it does offline.

GPL-3.0-or-later

4.6 Train

The Train screen is the loop that builds a training set: feed a case, capture it, label it, save it — and, when you have enough images, train a model from them.

4.6.1 When Train isn't available

Train is always in the sidebar, but it needs a local model of *your own* to work on. When the active model isn't one, the button is dimmed and the page says which of the two cases you are in:

- **AI Config mode** — classification is running over HTTP, so there is no model on this machine to train. Activate or create a local model on the Models page.
- **A community model** — the checkpoint belongs to its publisher and they keep updating it. To build on it, export it from the Models page and import the archive back as your own model.

Either way the page carries a button straight to the model library.

The same holds the other way round for AI Config, which is dimmed whenever a local model is active: clicking it opens its own page, where — in place of the server form — a panel names the model doing the classifying and offers the same jump to the model library. Both buttons stay in the sidebar in every mode — what changes is which one is live.

4.6.2 Capturing images

The left column reads top to bottom in the order you work:

1. The **case image** — the cropped headstamp of whatever was last fed.
2. **Feed**, centred beneath it — drops the next case and captures it. It needs the board connected.
3. **Label** and **Save image** — the label to file this image under, and the button that writes it. The label box remembers every headstamp the model knows, and you can type a new one.
4. The readouts — the predicted label (when a trained checkpoint and PyTorch are both present), how long image processing took, how long the prediction took, and a per-step breakdown. Useful for spotting a setup that has become slow.

PyTorch is *offered* here, never required: capturing and labelling images is exactly the work you do before there is anything to predict with. Declining costs you only the predicted-label convenience, and you won't be asked again this session.

4.6.3 Image counts

The right half lists every headstamp and how many images are on disk for it — your training set, straight from the folder. Drag the divider to give it more width and the list reflows into more columns, which is what makes a model with a hundred-odd headstamps readable.

Clicking a headstamp in this list saves the captured case under that label and immediately feeds the next one — the fastest way to work through a tray of mixed brass.

4.6.4 Training a model

The **Training** strip at the foot of the page turns the images above into a model:

- **Sort while training** routes each case you label into its own bin instead of the catch-all, using the label *you* picked — so a training session also sorts the tray.
- **Training settings...** opens the hyperparameters (epochs, batch size, learning rate, image size, focal loss, SWA and the rest). The defaults are sensible; change them when you know why. The ConvNeXt size is the model's own property — change it in the model editor, not here.
- **Start training** launches the run in a separate process and opens a live console for it. Closing the console mid-run asks first, and confirming cancels the run — the console is the only thing it reports to.

GPL-3.0-or-later

The console **opens with what the run is using**: the PyTorch version, whether CUDA was found, which GPU (and how much memory it has) or that it is training on the CPU, then every setting the run was given. That block is the first thing to quote when a run is slower than expected or a result looks wrong.

Every run is also written to a file, so the console closing doesn't lose it: `training-<date>.log` in the `logs` folder under your data folder (**File → Open data folder**), and the console's first line says exactly where. The last few runs are kept. Two shortcuts for sending one on:

- **Save log…** on the console writes what you are looking at wherever you choose;
- **Help → Export support package…** puts the most recent run's log in the ZIP as `training.log`, with your folder paths replaced by placeholders.

Training needs PyTorch. If it isn't installed, the app offers to install it here.

4.7 AI Config

The alternative to a local model: classification is sent to an OpenAI-compatible HTTP server, and this screen is where that server — and the headstamps it may answer with — is set up. It is Train's mirror in the sidebar: live whenever classification runs over HTTP, dimmed when a local ConvNeXt model does the work.

Two things can put classification on HTTP, and this page serves both:

- **AI Config mode** — no active model at all. The screen edits the app-level server settings.
- **An active OpenAI model** — a model whose training mode is **OpenAI** on the Models page. Each OpenAI model carries its **own** server settings and its own headstamp list, so several can coexist — different cartridges, prompts, even different providers. While one is active this screen edits *that model's* settings, and a caption above the form says so by name. This mirrors the Windows app, where "OpenAI API" is a Training Mode and its configuration lives on the model.

4.7.1 When AI Config isn't the one classifying

Activate a local ConvNeXt model and this screen swaps its form for a panel naming the model that is classifying instead, with a button straight to the Models page. Select **Use AI Config** — or an OpenAI model — there to come back; the server settings are still exactly as you left them.

4.7.2 Setting up the server

- **Server** — endpoint URL, API key, model name, the prompt (use `{{headstamps}}` where the list of headstamps should be injected), and the JPEG quality and scale used for the image. Press **Save** to apply — these fields do not save as you type.
- **Headstamps** — the list this mode routes on, and each one's slot. Add, rename, remove, clear, or **Load from server**. These do save immediately, and editing a slot here updates the Sort dashboard's cards straight away.
- **Test shot**, under *Test (capture → crop → classify)* — takes one frame from the camera, runs it through the same pipeline, and shows the label and confidence that came back. No board feed needed; it reads the fields above as they stand, so an API key and model name must be filled in.

4.8 Models

The model library. Everything the app knows how to classify with is a row in this table, plus one synthetic row for AI Config mode.

4.8.1 The model list

The table lists each model's name, whether it is active, its cartridge, type (yours or a community model), the mode it was built as — a ConvNeXt size, or **OpenAI** for a model that classifies over an HTTP server — how many training images it has, whether it has been trained, and when. Click a column heading to sort by it. An OpenAI model has nothing to train: activate it and the AI Config page becomes the place its server settings and headstamps live.

GPL-3.0-or-later

The **Active** column marks the model the app currently classifies with: exactly one row reads ● **ACTIVE** in the theme's action colour, and every other row's Active cell is blank. To change it, select a row and press **Activate** at the bottom right.

Above the table: filters by **cartridge** and **type**, a search box, and **New cartridge**, **New model** and **Import…**.

The **"Use AI Config"** row sits at the top of the list, whenever the filters and the search box leave it there. Activating it puts the app in AI Config mode — classification goes to the HTTP server configured on the AI Config screen instead of a local model. It is not a model, so Edit, Delete, Export and the rest stay greyed out while it is selected.

4.8.2 Managing a model

Everything on the bar under the table acts on the **selected** row. **Delete** sits alone on the far left and **Activate** on the far right, so the destructive one and the primary one can never be neighbours:

- **Delete** — delete the model and its folder, after a confirmation. The active model refuses: activate something else first.
- **Edit…** — name, cartridge, model size, primer settings, and the community feedback opt-in where it applies.
- **Images…** — browse the model's training images as thumbnails, and reclassify or delete them. A single image opens full size with prev/next navigation. Models you own only.
- **Headstamps…** — the model's headstamp list: add, rename, delete, set each one's slot, and group headstamps under parent classifications. Renaming a headstamp also renames its training images, so the label and the files stay in step.
- **Evaluate…** — score the model against a folder of labelled images and write an interactive HTML report. Only open reports you generated yourself, from image folders you trust.
- **Export…** — write the model out as a ZIP (checkpoint, headstamps and images), the format the Windows app uses.
- **Activate** — make this the model the app classifies with. Greyed out on the row that is already active.

4.8.3 Importing and exporting

Import… reads a model ZIP back in, after a notice that a model file can execute code — import only from authors you trust. If the archive carries a community ID you already have installed, the app asks whether to **update the installed one in place** — keeping its slot assignments, sorting templates and your name for it — or to **import it as a separate copy**; anything else lands as a new model. A ZIP you import stays *yours*: importing your own model onto a new machine leaves it trainable. Its trained model file still came from outside this machine, though, so it needs an up-to-date PyTorch before it loads (see "Update PyTorch to use this model").

An archive is refused if any image in it has a name that could point outside the model's folder — a drive letter or a colon in the name. Archives from this app and from the Windows app never have such a name.

Import and export both run in the background; a model with its images can be large.

4.9 Community

Published models, shared by other users. Signing in is the only thing in the app that needs an account — everything else works signed out.

The table lists each model with its cartridge, version, what the archive includes (model, images, or both), headstamp and image counts, size, publish date, author, and its **State** against your library: Available, Update available, or Installed.

GPL-3.0-or-later

Select a row and the two buttons under the table follow its state. The right-hand one is the primary, and it says what the state makes possible:

Button	Action
Download model	Download and install it, after a notice that a model file can execute code — download only from authors you trust.
Update model	Update your installed copy in place, or take this version as a separate copy; it asks which.
Already installed	Nothing to do — this version is the one you have.
Remove	On the far left, and live only for an installed, current model: delete your local copy. The catalogue entry stays, and the primary goes back to **Download model**.

Select a second model and press Download while one is still running and it queues behind the first, with the status showing "(2 of 3)" as it works through them. Selecting a queued model shows where it sits in the queue, and the one being fetched reads **Downloading…**.

A model you install this way is **managed by its publisher**: it can't be trained here, and updates from them install over it cleanly. The Sort screen tells you when a newer version exists — see community model notices.

Share a model… publishes one of your own models to the community. It appears only for accounts the server grants the contributor role. Sharing does not make the model foreign: your copy stays yours and stays trainable.

4.10 Settings

The Settings activity groups everything you configure once and rarely touch again, behind a section list. Most settings here save as you change them, with no separate Save step. The exception is the Camera page, where the device and resolution are committed by **Apply**.

4.10.1 Camera

Picks which camera the Sort dashboard's preview and the classifier both read from.

- **Detect / refresh** probes attached cameras (opening each one briefly to read its supported resolutions) and fills the **Camera** and **Resolution** dropdowns.
- **Apply** swaps the live camera to the selected device and resolution — the preview beneath updates immediately, so a bad pick is obvious before you leave the page.

Nothing here grabs a camera on its own; both actions are yours. The current device and resolution are shown beneath the controls.

GPL-3.0-or-later

4.10.2 Serial

Connects the app to the sorting machine over the board's UART protocol.

- **Port** — pick a detected serial port, or **Emulated** to run against the built-in board emulator with no hardware attached. Every detected port is listed here, but the automatic probe at startup is choosier: on macOS it skips Bluetooth and debug pseudo-ports (they can never be the board, and opening one can wake a paired headset), naming them in the serial monitor. A port you picked once is always probed, and connecting manually from here works for any port.
- **Baud** and **probe timeout** — connection parameters; the defaults match the firmware. The baud picker is shared with the Serial Monitor's.
- **Connect / Disconnect** open or close the link; the status bar's serial indicator mirrors the result.
- **Refresh ports** re-scans, for a board plugged in after the app started.
- **Open serial monitor ↗** reveals the Serial Monitor panel.
- **Initialize these settings on startup** pushes the board init settings below automatically on every connect, instead of only when you press "Push to board".
- **Log serial traffic to a file** writes everything the Serial Monitor shows (each line sent, each line received, and the app's own notes) to a file. Each line is timestamped to the millisecond. The file is `logs/serial-<date-time>.log` in the data folder (**File → Open data folder**), with one file per session. Ticking or unticking it takes effect immediately, with no restart and no reconnect. Leave it off for everyday sorting and turn it on when you are chasing a board problem or about to report one. The newest file goes into the support package automatically. The logs can't fill the disk: a file that reaches 5 MB is continued in a new one, and each new file clears out all but the ten before it.

Board init settings covers the machine's tunables — feed and sort speed, homing offsets, motor current, debounce timing and the camera LED level — with **Get config from board / Push to board** to read or write them. The **Sort arm** group jogs the arm to a slot or homes it, for testing wiring before a real run. **Airdrop configuration** holds the three timing values (pre-drop delay, signal duration, post-drop delay), plus the switch that turns it on, for boards fitted with an airdrop mechanism.

4.10.3 Image Processing

Tunes how a captured frame becomes the 480×480 headstamp image the classifier sees. **Capture** takes a frame and shows it beside the processed result, and every detection or primer change re-processes that same frame — so you tune against one case instead of re-feeding for every adjustment.

These settings belong to the active model, not to the app: case diameter and primer size are properties of the cartridge, so switching models brings its own values back. A model that has never been tuned inherits whatever is currently set, so nothing is lost when you activate one for the first time.

- **Configuration** — the circle-detection parameters: accumulator scale, minimum separation between centres, edge strength, detection threshold, and the minimum and maximum case radius in pixels. Start with the two radius values: they bracket the size of the case in the frame.
- **Primer mask** — leave the primer alone, keep only the primer area, or hide it, plus the primer radius. Hiding the primer helps when primer markings confuse the classifier.
- **Camera LED brightness** — the board's ring-light level. This one *is* global: it is a property of the machine, not of the model. The board is updated shortly after you stop moving the slider.

4.10.4 Theme

Picks the colour theme. The same list is in the Themes panel, which is the easier place to try several.

Edit theme… opens the theme editor: it starts from the theme you are currently using and gives you a colour picker per role with a live preview. **Save & apply** writes back to a theme you made (renaming it moves it rather than copying), makes a new one when you started from a built-in, and leaves the editor open so you can keep adjusting — **Close** ends the session; **Create new…** saves under a name you pick, and a built-in is never overwritten either way. A theme can also be exported to a file and imported on another machine.

GPL-3.0-or-later

4.10.5 Import from Windows

Copies your setup out of an installation of the Windows app (**AI Brass Sorter**) so you do not have to build it again here. **Nothing in the Windows app is changed, moved or deleted** — everything is copied, and that installation keeps working exactly as it did.

If the Windows app is installed in the usual place, this page finds it on its own; otherwise choose the folder yourself — the one containing `Data` and `training`. The same offer appears once, automatically, the first time you start this app on a computer that has the Windows app on it.

You tick what comes across, in a tree:

- **Models** — with one branch per model in the Windows app. Under each model:
- **Training images** — usually the bulk of the data, and the slowest part of the copy.
- **Headstamps and slot assignments** — that model's headstamp list, parent classifications, and which bin each one drops into.
- **Trained model file** — its size is shown, because this is the big one. Take it and the model classifies immediately; leave it and the model comes across as a shell you retrain on the Train page.
- **Image-processing settings** — the crop tuning from the Windows app.
- **Serial / board settings** — port, baud rate and the board's init values.
- **AI Config** — endpoint, model and prompt, if you classify over HTTP.

Pick the models you actually want. A Windows install that has been in use for a while usually holds models you have no interest in carrying forward, so every model is its own tick. **Select all / Select none** are there so choosing two out of fifteen is two clicks rather than thirteen, and the line under the tree totals what you have chosen — models, images and trained model files — before you start.

Images, headstamps and the trained model file all belong to a model, so they sit *under* it: untick a model and its whole branch goes with it. There is no way to bring a model's images across without the model itself, because they land in its folder. A model showing a half-filled tick is one where you have kept some parts and not others.

Anything this installation does not have is left out or greyed out rather than offered as a tick that would import nothing — a model with no images has no **Training images** row at all.

Each model's row also says what importing it would **do to your library here**: either *new model here*, or *updates* '*...*' naming the model it would refresh instead of duplicating.

A few things are worth knowing before you run it:

- **Models trained with the Windows app's older ML.NET pipeline cannot classify here.** They are still imported — with their headstamps and images — but without a trained model file, so the Train page is where you pick them up. Their row says *ML.NET model, retrain needed* and has no **Trained model file** to tick, so you can decide before you import; the summary at the end names the ones you did import.
- **A model that came from the Community stays read-only**, the same as one downloaded here: the trained model file belongs to whoever published it, so it is not trainable. Your own models stay trainable. If you already downloaded that model here, the import updates your copy and it stays read-only.
- **Trained model files imported from the Windows app need PyTorch 2.10.0 or newer** before they load, like any model file that wasn't trained on this computer (see "Update PyTorch to use this model").
- **Board settings that aren't whole numbers are skipped.** The board only takes whole numbers, so any other value stays at this app's default.
- **A Windows model set to classify over an OpenAI server imports as an OpenAI model here too**, keeping its own endpoint, prompt and headstamps. Activate it and the AI Config page edits its settings.
- **One model that cannot be imported does not stop the rest.** It is skipped, the reason is listed at the end, and everything else still lands.

GPL-3.0-or-later

Running the import a second time is safe. A model already brought across is updated in place rather than duplicated, your slot assignments and sorting templates survive, and images already copied are skipped. The import never takes over an active model you have already chosen here.

That also means you can import in passes — bring two models over, sort with them, then come back for more. And a model whose trained model file you declined the first time gets it on a later run if you tick it then.

If a Windows model happens to share its name with a model you already have here, the imported one is named `… (2)` rather than leaving you with two rows you cannot tell apart. Your existing model is not touched.

4.11 Getting help

4.11.1 The guide

`F1`, or **Help → User Guide**, opens this guide in a panel at the section for the screen you are on. It is a normal panel: dock it beside your work while learning something, close it after.

4.11.2 Support package

Help → Export support package… collects your current configuration into a plain-text report — the app version and platform, the active model, run options, training configuration, image processing, serial and camera settings, and the AI Config setup.

Copy to clipboard gives you the text to paste on the community Discord when asking for help. **Save package…** writes a ZIP holding the same report, a machine-readable `config.json`, and — if you have trained a model — the most recent run's log as `training.log`. If you have logged serial traffic, the newest serial log goes in as `serial.log`.

It is safe to share: the API key is reported only as "set" or "not set", nothing is read from the sign-in cache, and file paths are shown relative to the data folder, so your home directory never appears. Both logs get the same treatment: your folder paths are replaced by `<data>`, `<app>` and `<home>` before they go in.

4.11.3 Updates

The app checks for a new release shortly after it starts and, if there is one, offers it in the status bar. **Help → Check for updates…** asks immediately.

The dialog shows the release notes and a **Download & install** button. Downloading only stages the update — nothing is replaced until you restart, and the button becomes **Restart now** when it is ready. **Choose a different version…** lists every published release, including older ones, and optionally pre-releases (which the automatic check never offers). You can turn the automatic check off in the same dialog.

When GitHub publishes a SHA-256 checksum for a release, the download is checked against it. A line under the release summary says whether it will be, and once the update is downloaded, whether it was:

- **The download is checked against the SHA-256 checksum…** — before downloading; once it is staged this becomes **Verified**.
- **This release has no published checksum…** — it still installs, but only the HTTPS connection vouches for it. Once staged this reads **Not verified**. A checksum in a format this version can't check is treated the same way, and names the format.
- **This release's published checksum can't be read…** — the release can't be installed from here; **Download & install** stays disabled.

A download that doesn't match its checksum is thrown away and nothing is staged; the dialog offers **Try again**. If it fails the same way every time, see *An update fails its checksum* on the Troubleshooting page.

GPL-3.0-or-later

5. Troubleshooting

Problems with the **application**. Mechanical faults — jams, a feed wheel that overruns its homing sensor, motors losing position — belong to the machine and its own documentation; what this page can do for those is show you the traffic (Serial Monitor) and the settings that drive them (Settings → Serial).

Before reporting anything, take **Help → Export support package…** — it collects your version, platform, active model, run options and every settings page into one report, with the API key reported only as "set" or "not set" and paths relative to the data folder. Attach the logs described below too.

5.1 Where the logs are

They live in the data folder's `logs/` directory — on Windows that is `%LOCALAPPDATA%\CaseSorter\logs`, on Linux and macOS `~/.local/share/CaseSorter/logs`. They answer different questions:

- `casesorter.log` — the application itself. It keeps several launches of history (rotating at 1 MB, three files back), so a problem you only got round to reporting after restarting a few times is still in there. This is the one to attach to a bug report.
- `launch.log` — the launcher: finding Python, syncing dependencies, applying an update, and anything the app printed. Rewritten on every start, with the previous one kept as `launch.prev.log`.
- `serial-<date-time>.log` — every line sent to and received from the board, timestamped. Only written while **Log serial traffic to a file** is ticked on Settings → Serial. It is off by default.

Reporting a board problem? Tick **Log serial traffic to a file** first, reproduce the problem, then export the support package. The package's ZIP carries the newest serial log as `serial.log`. The Serial Monitor only holds what is on screen, while the file keeps the whole session, including the moments before you noticed.

To collect more detail, start the app with `CASESORTER_LOG_LEVEL=DEBUG` set in the environment.

5.2 An error flashed past in the status bar

Click the status bar's message, or open **View → Messages**: the Messages panel keeps the last 200 status lines in full, errors marked, and **Copy all** puts them on the clipboard for a report.

5.3 Nothing happens when I start the app

Read `launch.log` — it holds everything from the launcher onwards, including the traceback. On Windows the console closes with the process and takes the error with it, which is what "nothing happened" usually is.

The first launch after an install or an update also syncs dependencies, which takes minutes rather than seconds. That is in the log too.

5.4 The window won't open on Linux

If the log ends with Qt failing to load the **xcb** platform plugin, install `libxcb-cursor0` (`xcb-util-cursor` on Fedora and Arch) — see Install for the full table. Without it Qt falls back to Wayland, where floating panels can't be moved or resized; without `libGL` or `glib` the app can't start at all.

GPL-3.0-or-later

5.5 The sorter is not detected

Almost always something else is holding the serial port, or the port never appeared:

- **Close the Arduino IDE.** Its serial monitor keeps the port open and the app can't have it. Any other terminal on the port does the same.
- **Press Refresh ports** on Settings → Serial — a board plugged in after the app started isn't in the list until you rescan.
- **Re-seat the USB cable**, try a different port, and bypass any USB hub.
- **On Linux, check permissions.** A port you can see but can't open is usually group membership — `dialout` on Debian/Ubuntu, `uucp` on Arch — and the change takes effect on next login.
- **Watch the handshake.** Open the Serial Monitor; it replays what the failed connect attempt actually said, which distinguishes "no port" from "port opened, board didn't answer".

To keep working meanwhile, choose the **Emulated** port: everything except moving brass runs.

5.6 The camera preview is black

- Press **Detect / refresh** on Settings → Camera, pick the device again and press **Apply** — the selection isn't live until Apply.
- **Close anything else using the camera.** One process at a time.
- **Try a lower resolution.** Not every mode a camera advertises actually streams.
- On the Sort dashboard, **Show live camera** is off by default and fetches no frames while off — an empty spot there is not a fault.

5.7 The crop is wrong, or the case isn't found

Tune it against a single frame on Settings → Image Processing: **Capture** once, then adjust — every change re-processes that same frame.

Start with the **minimum and maximum case radius**, which bracket the size of the case in pixels; a case outside that bracket is simply not detected. If primer markings are confusing the classifier, **hide the primer**.

Remember these values belong to the **active model**, so a model trained with one crop and run with another will classify badly even though nothing looks broken. Tune once, then train.

5.8 Everything lands in the Catch-All

Slot 0 takes anything unrouted, unrecognised, or below the confidence floor. In order of likelihood:

1. **The headstamps aren't routed.** Click a slot card and tick them — nothing is assigned on a fresh layout.
2. **The confidence floor is too high** for this model. It's in ⚙ **Run options**; the per-case confidence shown under the crop tells you what the model is actually scoring.
3. **The crop doesn't match what the model was trained on** — see above.
4. **The model is the wrong one.** Check which row reads ● **ACTIVE** on Models.

5.9 Start is greyed out or refuses

It always says which one it is: no board connected, the API key or model name unset in AI Config mode, the active model's checkpoint missing, PyTorch not installed yet for a local model, or an unread moderator note.

A **missing checkpoint** means the model row exists but its `.pth` doesn't — usually a community share that carried images only, or a data folder that moved. The app will not quietly fall back to sending your images to an HTTP server; pick a different model, or train this one.

GPL-3.0-or-later

5.10 Training fails or the machine runs out of memory

- **PyTorch isn't installed** — the app offers to install it when you press Start training.
- **Out of memory:** lower **batch size** or **image size** in **Training settings…**, and close other applications. Training on CPU works and is slow; a GPU below compute capability 8.0 is not used.
- **The first training run may need the network** to fetch pretrained weights.
- **Label mismatches:** training images are named `{label}__{ticks}.jpg` and the label is case-sensitive. Renaming files by hand outside the app is how a class ends up split in two — use **Images…** on the Models screen to reclassify instead.
- The live console is the only place the run reports to; closing it cancels the run, after asking.

5.11 The app says a model update is available every time

The status-bar notice is the *model's* publisher releasing a new version, not an app update. **Update now** installs it in place and keeps your slot assignments, sorting templates and the name you gave it. Dismissing it is fine — it returns next time you open the Sort screen.

5.12 An update fails its checksum

The update dialog says the download *does not match the SHA-256 checksum GitHub published for it*. The file was discarded and nothing was staged or installed — the app you are running is untouched.

Press **Try again**. A truncated or corrupted download is the usual cause, and a retry over a steadier connection fixes it. If it fails the same way every time, something between you and GitHub is changing the file: don't install that release by hand, and report it with `casesorter.log` attached — it records the checksum that was expected and the one that arrived. The Windows installer applies the same check and stops the same way; its `install-<timestamp>.log` holds the same two values.

5.13 Resetting

Deleting the data folder (**File → Open data folder**) resets the app to a fresh install — settings, models, training images and all. There is no undo, so export anything you want to keep first.

GPL-3.0-or-later